

Retail Vision Intelligence System: Sistema de Inspeção Visual de Prateleiras com LLM Multimodal, Regras em Linguagem Natural e Memória RAG

João Nuno Fonseca da Silva – Nº 51557

Licenciatura em Inteligência Artificial e Ciência de Dados

Interação com Modelos de Larga Escala

Ano letivo 2025/2026

Repositório GitHub: github.com/JoaoNuno2005/TRB_IMLE

Resumo—Este relatório descreve o desenvolvimento do *Retail Vision Intelligence System*, um sistema de inspeção visual de prateleiras para contexto de retalho. O objetivo foi complementar a inteligência operacional baseada em dados estruturados do Projeto 1 com uma camada visual capaz de analisar imagens de prateleiras, identificar falhas operacionais, executar regras definidas em linguagem natural pelo gestor e recuperar contexto histórico através de uma memória vetorial. A solução foi organizada em cinco componentes independentes: `shelf_inspector.py`, `rule_engine.py`, `rag_memory.py`, `report_generator.py` e `interface.py`, complementados por `evaluate.py`, `evaluate_judge.py`, uma interface *Streamlit*, cache local, prompts versionados, persistência de regras e inspeções, e uma *vector store*. O sistema usa modelos Gemini Flash parametrizados por variáveis de ambiente, inclui modo *mock* para demonstração robusta, e foi desenhado para funcionar mesmo quando a quota da API está indisponível. O relatório detalha arquitetura, dataset, estratégias de *prompting*, RAG, regras, avaliação, limitações e decisões de engenharia.

Index Terms—LLM multimodal, inspeção visual, retalho, RAG, ChromaDB, Gemini Flash, prompt engineering, rule engine, avaliação automática.

I. INTRODUÇÃO E MOTIVAÇÃO

O Projeto 1 produziu inteligência operacional a partir de dados estruturados de uma loja, nomeadamente sequências temporais de presença, zonas visitadas, tempos de permanência e anomalias de tráfego. Essa abordagem permite compreender a circulação dos clientes, mas não observa diretamente o estado físico das prateleiras. Uma zona pode apresentar elevada afluência e, simultaneamente, estar a perder vendas por falta de reposição, produtos deslocados, prateleiras desalinhas ou incumprimento de planograma. O Retail Vision Intelligence System foi desenvolvido para preencher essa lacuna através de uma camada de inteligência visual.

O sistema recebe imagens de prateleiras, envia-as para um modelo multimodal, produz uma inspeção estruturada em JSON, aplica regras operacionais previamente definidas em linguagem natural, indexa o resultado numa memória RAG e gera relatórios executivos em Markdown. A solução foi

construída numa perspetiva de aluno de Inteligência Artificial e Ciência de Dados: mais do que chamar uma API, o trabalho procurou demonstrar desenho de arquitetura, engenharia de prompts, persistência de estado, avaliação, tratamento de erros e preocupação com reprodutibilidade.

As principais contribuições implementadas foram:

- uma pipeline modular de inspeção visual, com entradas e saídas explícitas em cada componente;
- três estratégias de *prompting*: *zero-shot*, raciocínio visual estruturado e *few-shot* textual;
- cache local por hash MD5 da imagem, para reduzir chamadas repetidas à API;
- mecanismo de *rate limiting*, *backoff* e *fallback* quando a API não está disponível;
- *rule engine* capaz de converter regras em português para JSON executável;
- memória RAG local com *embeddings* multilingues e persistência em ChromaDB;
- geração automática de relatórios com sumário executivo, problemas por zona, regras disparadas, histórico e recomendações;
- interface CLI e interface visual em *Streamlit*, pensada para simular um gestor de loja real;
- avaliação com `evaluate.py` e avaliação qualitativa com `evaluate_judge.py`.

II. ARQUITETURA DO SISTEMA

A arquitetura segue a decomposição exigida no enunciado: cada módulo executa uma responsabilidade clara e comunica através de objetos JSON persistidos em disco. Esta decisão reduz o acoplamento, facilita a defesa oral e permite testar cada componente isoladamente.

A. Estrutura do repositório

A estrutura final do projeto foi organizada para coincidir com os entregáveis pedidos e para suportar execução local em macOS com ambiente virtual Python. A pasta principal

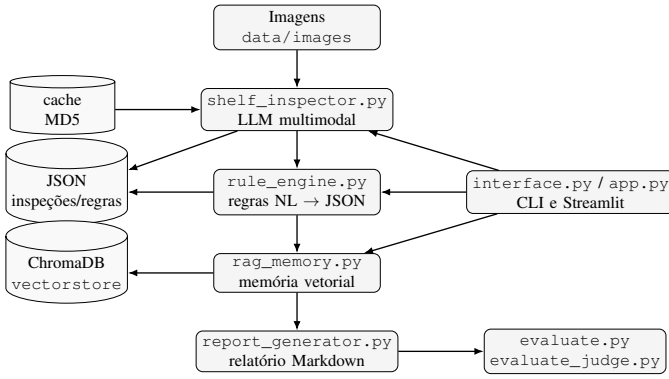


Figura 1. Arquitetura modular do sistema implementado.

contém o código-fonte, os dados, os prompts e os artefactos gerados em runtime.

Listing 1. Estrutura funcional do projeto.

```
tp2_novo/
|-- README.md
|-- requirements.txt
|-- .env.example
|-- app.py
|-- evaluate.py
|-- evaluate_judge.py
|-- cache/
|-- vectorstore/
|-- prompts/
|   |-- visual_zero_shot.txt
|   |-- visual_cot.txt
|   |-- visual_few_shot.txt
|   |-- rule_parser.txt
|   |-- rag_answer.txt
|   |-- report_generator.txt
|-- data/
|   |-- images/
|   |   |-- normal/
|   |   |-- empty_shelf/
|   |   |-- wrong_product/
|   |   |-- dirty_misaligned/
|   |   |-- ambiguous/
|   |-- uploads/
|   |-- inspections/
|   |-- rules/
|   |-- reports/
|-- src/
|   |-- shelf_inspector.py
|   |-- rule_engine.py
|   |-- rag_memory.py
|   |-- report_generator.py
|   |-- interface.py
```

B. Decisões de desenho

A primeira decisão foi manter os componentes independentes. O `shelf_inspector.py` não conhece detalhes da interface; apenas recebe imagem, zona e estratégia de prompt, devolvendo uma inspeção estruturada. O `rule_engine.py` não processa imagem; recebe inspeções e regras persistidas. O `rag_memory.py` não decide severidade; apenas indexa e recupera contexto. O `report_generator.py` combina inspeções, regras e histórico para criar um relatório operacional.

A segunda decisão foi usar ficheiros JSON e Markdown como formato intermédio. Isto torna os resultados auditáveis: é possível abrir uma inspeção em `data/inspections`, verificar os *issues*, consultar que regras dispararam e reconstruir

Tabela I
DISTRIBUIÇÃO FINAL DO DATASET POR CATEGORIA OPERACIONAL.

Categoria	Mín.	Final	Dif.
Normal	150	167	+17
Prateleira vazia	100	109	+9
Violação de planograma	100	100	0
Suja/desordenada	80	96	+16
Ambígua	70	73	+3
Total	500	545	+45

o relatório. Esta transparência é importante porque sistemas com LLMs podem produzir saídas plausíveis mas incorretas; guardar o estado permite inspeção humana e repetição dos testes.

A terceira decisão foi tornar a API configurável. A chave Gemini nunca é colocada no código; é lida de `.env`. O modelo também é parametrizável, permitindo usar um modelo Flash compatível com visão, como `gemini-1.5-flash` ou outro modelo Flash disponível na conta. Para testes e defesa, foi ainda incluído `TP2 MOCK_LLM=1`, que permite correr a pipeline sem depender da quota externa.

III. DATASET

A. Origem e composição

O dataset foi construído a partir de fontes públicas de retalho, tendo como fonte principal o SKU-110K, que contém imagens densas de prateleiras de supermercado e é adequado para problemas de deteção em cenários com muitos produtos visualmente semelhantes [2]. Como complemento conceptual, foram consideradas fontes como o Grocery Store Dataset, que disponibiliza imagens naturais de produtos recolhidas em lojas e organizadas por classes finas e grossas [3]. A seleção final foi organizada localmente nas cinco categorias exigidas: prateleira normal, prateleira vazia, violação de planograma, prateleira suja/desordenada e caso ambíguo.

Na prática, a organização local foi feita através de subpastas em `data/images/`. A contagem final classificada foi de 545 imagens: 167 normais, 109 de prateleira vazia, 100 de violação de planograma ou produto errado, 96 de prateleira suja/desordenada e 73 casos ambíguos. Assim, todas as categorias cumprem os mínimos definidos no enunciado e o dataset excede o requisito global em 45 imagens. A pasta `data/uploads/` foi separada das classes de dataset para evitar que imagens submetidas pela interface fossem acidentalmente confundidas com dados classificados. Também foram removidas inspeções antigas marcadas com `Z_UNKNOWN`, para garantir consistência entre zona, imagem e histórico.

B. Critério de classificação

A classificação inicial das imagens não foi tratada como verdade absoluta. Em vez disso, cada imagem foi observada segundo um critério operacional:

- se a imagem apresenta uma prateleira cheia, alinhada e sem anomalia evidente, entra em `normal`;

- se há espaços vazios visualmente relevantes, entra em `empty_shelf`;
- se o problema principal é posicionamento errado, etiqueta ausente, produto tombado ou provável incumprimento de planograma, entra em `wrong_product`;
- se a imagem mostra desalinhamento forte, desarrumação ou dano, entra em `dirty_misaligned`;
- se a evidência não permite decisão segura, entra em `ambiguous`.

Este critério é coerente com a natureza do problema: o objetivo não é reconhecer SKU individual com precisão de inventário, mas detetar situações operacionais que exigem atenção do gestor.

C. Licenças e limitações

A utilização de imagens públicas foi documentada por fonte. O SKU-110K é usado como referência de imagens de prateleiras densamente povoadas, e o Grocery Store Dataset apresenta licença MIT no respetivo repositório público [3]. Quando uma imagem provém de fonte externa, a origem deve ser mantida num manifesto local, por exemplo `dataset_manifest.csv`, com campos `filename`, `source`, `url`, `license`, `category` e `notes`. Esta prática é relevante para a integridade académica e para a defesa oral.

As principais limitações do dataset são:

- **viés visual**: datasets públicos podem ter iluminação, perspetiva e organização diferentes das lojas portuguesas reais;
- **rotulagem aproximada**: as classes operacionais foram adaptadas a partir de imagens que nem sempre foram originalmente anotadas para *planograma*;
- **ausência de ground truth fino**: muitas imagens não têm anotação manual por zona da prateleira;
- **ambiguidade**: certos casos dependem de contexto de loja, por exemplo saber se um produto está realmente na posição errada;
- **generalização**: a avaliação do professor poderá usar imagens não vistas, com produtos, iluminação e ângulos diferentes.

Estas limitações influenciam a avaliação: métricas como *Issue Detection Rate* são úteis, mas devem ser interpretadas com análise qualitativa de falhas, sobretudo em casos ambíguos.

IV. COMPONENTE 1: ANÁLISE VISUAL

A. Modelo e interface com API

O módulo `shelf_inspector.py` é responsável por transformar uma imagem de prateleira num registo estruturado. O sistema foi desenhado para usar Gemini Flash através da API, porque os modelos Gemini suportam geração multimodal e saída estruturada; a documentação oficial descreve suporte a entrada multimodal e a *structured outputs* com JSON Schema [4]. A opção por modelos Flash decorre da latência, custo e disponibilidade em conta gratuita ou de baixo custo. A documentação de *rate limits* indica que os limites dependem do projeto, do modelo e do escalão da conta, sendo medidos em dimensões como RPM, TPM e RPD [5]. Por isso, o sistema não assume disponibilidade ilimitada.

O output obrigatório segue o schema do enunciado. Cada inspeção contém identificador, data, caminho da imagem, zona, estado global, lista de problemas, *fill rate*, produtos visíveis e campo de raciocínio visual. Um exemplo simplificado é apresentado na Listagem 2.

Listing 2. Exemplo de *inspection record* simplificado.

```
{
  "inspection_id": "INS_20260612_170901_001",
  "timestamp": "2026-06-12T17:09:01Z",
  "image_path": "data/images/normal/train_1.jpg",
  "zone_id": "Z_S1",
  "overall_status": "warning",
  "issues": [
    {
      "issue_id": "ISS_001",
      "type": "misaligned",
      "location": "middle shelf, right side",
      "severity": "medium",
      "description": "Some products appear misaligned.",
      "confidence": 0.72,
      "affected_area_pct": 12.5
    }
  ],
  "shelf_fill_rate": 0.86,
  "products_detected": ["packaged goods"],
  "model_reasoning": "Visual reasoning summary before final classification."
}
```

B. Validação e normalização

Como os LLMs podem devolver texto adicional antes ou depois do JSON, a implementação inclui extração e validação da estrutura. Quando a resposta não é parseável, o sistema tenta normalizar a saída. Se a API falhar ou a quota estiver indisponível, o sistema usa o modo *mock* ou devolve mensagem clara, sem expor stack trace ao utilizador final.

Foi também implementada persistência dos resultados em `data/inspections/`. Esta escolha permite que uma imagem previamente analisada seja reutilizada no relatório e no RAG sem nova chamada à API.

C. Estratégias de prompting

Foram implementadas três estratégias obrigatórias:

1) *Estratégia A: zero-shot direto*: O prompt pede ao modelo que analise a imagem e devolva apenas JSON válido. Esta estratégia é simples, consome poucos tokens e funciona bem para imagens óbvias. A sua limitação é a menor capacidade de decompor a cena por regiões; em casos ambíguos, o modelo pode saltar diretamente para uma classificação plausível sem justificar adequadamente a evidência visual.

2) *Estratégia B: raciocínio visual estruturado*: O prompt força uma sequência de análise: observar a imagem, dividir a prateleira por regiões, procurar lacunas, desalinhamentos, dano, produtos tombados e etiquetas ausentes, e só no fim construir o JSON. Esta estratégia tende a ser mais robusta em imagens complexas, porque obriga o modelo a relacionar a classificação com evidência visual. O custo é maior consumo de tokens e maior risco de respostas longas.

3) *Estratégia C: few-shot textual*: Como enviar múltiplas imagens como exemplos consumiria quota, os exemplos *few-shot* foram textuais: descrições de inspeções anteriores corretas

Tabela II
COMPARAÇÃO QUALITATIVA DAS ESTRATÉGIAS DE PROMPTING.

Estratégia	Vantagem principal	Falha típica
Zero-shot	Rápida, simples e econômica.	Menor detalhe em casos ambíguos.
CoT visual	Melhor cobertura região a região.	Mais tokens e maior verbosidade.
Few-shot	JSON mais estável e severidades mais consistentes.	Possível enviesamento pelos exemplos.

e o respetivo JSON esperado. Esta estratégia melhora a consistência do formato e ajuda o modelo a alinhar severidade e tipos de *issue*. A limitação é que os exemplos textuais podem enviesar a resposta quando a imagem atual difere muito dos casos exemplificados.

D. Gestão de quota

O sistema inclui três mecanismos obrigatórios. Primeiro, calcula o hash MD5 da imagem e guarda a resposta em cache, evitando chamadas repetidas para a mesma imagem não modificada. Segundo, aplica *rate limiting* e *backoff* quando recebe erros de quota. Terceiro, mantém *fallback* gracioso: com imagens em cache ou modo *mock*, a demonstração continua funcional mesmo sem acesso momentâneo à API.

V. COMPONENTE 2: RULE ENGINE

A. Objetivo

O `rule_engine.py` resolve a diferença entre a linguagem do gestor e a estrutura computacional necessária para executar regras. O gestor não deve escrever JSON nem conhecer nomes internos como `fill_rate_threshold`. Pode dizer, por exemplo, «avisa-me quando a prateleira inferior estiver mais de 40% vazia». O sistema converte essa frase para uma regra estruturada com condições, ação e validação.

B. Schema de regra

Cada regra persistida contém identificador, data, frase original, descrição formal, condições, ação e validação. A Listagem 3 mostra um exemplo.

Listing 3. Exemplo de regra estruturada.

```
{
  "rule_id": "RULE_001",
  "created_at": "2026-06-12T17:20:00Z",
  "natural_language": "Avisa-me quando a prateleira inferior estiver mais de 40% vazia",
  "description": "Gerar alerta quando a prateleira inferior tiver fill rate inferior a 60%.",
  "conditions": {
    "zone_filter": [],
    "time_filter": null,
    "issue_types": ["empty_shelf"],
    "severity_threshold": "medium",
    "fill_rate_threshold": 0.60,
    "location_filter": "bottom"
  },
  "action": {
    "alert_level": "warning",
    "notification_message": "Prateleira inferior com stock insuficiente na zona {zone_id}."
  },
  "validation": {
    "is_valid": true,

```

```
"ambiguities": [],
"assumptions": ["Aplicavel a todas as zonas."]
}
```

C. Detecção de ambiguidades

Um requisito importante é não assumir silenciosamente quando a regra é incompleta. A frase «avisa-me quando a prateleira estiver vazia» é ambígua porque não especifica percentagem, zona, período temporal nem urgência. O sistema deve devolver essas ambiguidades ao gestor antes de guardar a regra. Esta opção é essencial para evitar falsos alarmes: uma regra mal interpretada pode gerar notificações excessivas ou deixar de alertar em situações críticas.

A estratégia implementada combina prompt especializado e validação determinística. O LLM identifica os campos em falta, mas o código verifica se existem condições mínimas: tipo de *issue*, limiar aplicável, zona ou indicação de aplicação global, e nível de alerta. Quando `validation.is_valid=false`, a regra não deve ser executada como regra ativa.

D. Execução

Depois de cada inspeção, o executor percorre todas as regras em `data/rules/`. Para cada regra, verifica:

- se a zona da inspeção está no filtro da regra;
- se o horário da inspeção respeita o filtro temporal;
- se algum *issue* corresponde aos tipos definidos;
- se a severidade atinge o limiar mínimo;
- se o *fill rate* ultrapassa ou fica abaixo do limiar definido;
- se a localização do problema coincide com a localização filtrada.

O resultado é um log de execução: regras verificadas, regras disparadas, justificação e mensagem de notificação. Esta explicabilidade é importante porque permite ao gestor perceber por que motivo recebeu um alerta.

VI. COMPONENTE 3: RAG MEMORY

A. Motivação

Sem memória, cada imagem seria analisada em isolamento. No entanto, em retalho, o valor operacional surge muitas vezes da repetição: uma prateleira vazia uma vez pode ser casual; a mesma zona vazia várias tardes seguidas sugere falha de reposição, procura acima da média ou erro de planograma. O `rag_memory.py` indexa cada inspeção para permitir perguntas históricas em linguagem natural.

B. Modelo de embeddings e vector store

A implementação usa embeddings de texto com `sentence-transformers/paraphrase-multilingual-MiniLM-v1`, um modelo que mapeia frases e parágrafos para vetores densos de 384 dimensões e é indicado para pesquisa semântica [6]. A persistência é feita em ChromaDB, uma infraestrutura *open-source* para recuperação que suporta armazenamento de documentos, metadados, pesquisa por similaridade e filtros por metadados [7].

A decisão por uma solução local tem três vantagens: não expõe dados para outro serviço externo, funciona sem servidor adicional e permite demonstrar o sistema num computador pessoal. A pasta `vectorstore/` é gerada em runtime e pode ser apagada/reconstruída a partir dos registos em `data/inspections/`.

C. Estratégia de chunking

Foram consideradas três estratégias:

- **chunk por inspeção completa:** simples, mas mistura problemas diferentes num só vetor;
- **chunk por issue:** granular, mas aumenta o índice e pode perder contexto global;
- **estratégia híbrida:** indexar um resumo rico da inspeção e guardar metadados estruturados.

A solução adotada foi a híbrida. O texto indexado é um resumo semanticamente rico, contendo zona, data, *fill rate*, tipo de problema, severidade e localização. Os metadados guardam campos como `inspection_id`, `zone_id`, `timestamp`, `overall_status` e `shelf_fill_rate`. Esta solução mantém recuperação semântica útil e permite filtragem por zona ou período.

Listing 4. Exemplo de summary indexado no RAG.

```
Inspection INS_20260612_171000 in zone Z_S3. Shelf fill rate 0.58. Detected empty shelf on bottom-left area with high severity. Affected area 35%. Products visible: packaged goods and beverages. Rule RULE_001 triggered warning. Similar issue occurred in previous inspections of Z_S3.
```

D. Queries suportadas

A interface histórica aceita perguntas como:

- «Quando foi a última vez que a zona Z_S1 teve problemas de prateleira vazia?»
- «Que zonas tiveram mais *issues* de planograma nas últimas duas semanas?»
- «Existe algum padrão nos problemas detetados às sextas-feiras à tarde?»
- «Que regras foram mais frequentemente disparadas este mês?»

O sistema recupera os *top-k* registos relevantes e sintetiza uma resposta com referências explícitas aos `inspection_id` e datas. Isto reduz alucinação porque a resposta é condicionada pelos chunks recuperados.

VII. COMPONENTE 4: REPORT GENERATOR

A. Objetivo

O `report_generator.py` foi corrigido para cumprir as secções obrigatórias do enunciado. O relatório final é gerado em Markdown e deve ser útil para um gestor de loja, não apenas para um programador. Por isso, evita linguagem excessivamente técnica e privilegia ações executáveis.

As secções obrigatórias implementadas são:

- 1) sumário executivo com máximo de 150 palavras;
- 2) problemas por zona, com severidade, *fill rate* e comparação histórica;

- 3) regras disparadas e ação gerada;
- 4) contexto histórico relevante com `inspection_id` e data;
- 5) recomendações concretas, no máximo cinco, ordenadas por urgência;
- 6) integração com trajetória, quando existirem dados do Projeto 1.

B. Critérios de qualidade

O principal erro observado antes da correção era o risco de produzir um relatório genericamente correto, mas pouco alinhado com as exigências formais: recomendações vagas, histórico sem referência explícita ou ausência de distinção entre *warnings* e *critical issues*. A versão corrigida força a presença das secções e limita o sumário executivo.

O `evaluate_judge.py` foi usado para verificar automaticamente a presença das secções. Num teste local, quando a API não estava disponível, o avaliador funcionou em modo `local_fallback`, mantendo a estrutura de LLM-as-Judge e atribuindo pontuação máxima à presença das secções obrigatórias do relatório. Esta pontuação não deve ser confundida com avaliação semântica perfeita; ela confirma cobertura estrutural, mas a revisão humana continua necessária.

VIII. COMPONENTE 5: INTERFACE CONVERSACIONAL

O enunciado permite CLI ou Streamlit. A solução inclui ambas. A CLI é útil para avaliação e reprodutibilidade; a interface Streamlit é útil para demonstração, porque aproxima o sistema de uma ferramenta real de gestor de loja.

A. CLI

A CLI permite executar inspeções, consultar histórico, gerir regras e gerar relatórios. Exemplos:

Listing 5. Exemplos de utilização da CLI.

```
python src/interface.py inspect Z_S1 --image data/images/normal/train_1.jpg
python src/interface.py inspect all --images-dir data/images/normal
python src/interface.py add rule "Avisa-me quando a prateleira inferior estiver mais de 40% vazia"
python src/interface.py list rules
python src/interface.py history "ultima vez que Z_S1 teve fill rate abaixo de 50%"
python src/interface.py report --session today
```

B. Streamlit

O `app.py` apresenta uma interface visual com navegação por separadores: inspeção, regras, histórico, relatórios e avaliação. A intenção foi simular um painel de gestor de loja: o utilizador pode carregar imagem, escolher zona, executar análise, ver problemas, consultar regras disparadas e gerar relatório. A interface também evita mostrar stack traces ao utilizador final, substituindo erros técnicos por mensagens compreensíveis.

IX. AVALIAÇÃO

A. Harness executável

A avaliação quantitativa é central porque o sistema usa LLMs, e outputs plausíveis não bastam. O ficheiro `evaluate.py` foi criado para correr com um único comando:

```
python evaluate.py --images-dir test_images/ --output
evaluation_report.json
```

Durante a defesa, o professor poderá fornecer imagens não vistas. O harness deve processar essas imagens, comparar com ground truth e gerar `evaluation_report.json`. Como o conjunto de teste do professor não existe antes da defesa, os resultados locais devem ser interpretados como avaliação de desenvolvimento e não como garantia de generalização.

No ensaio local realizado sobre `data/images`, o harness correu com a estratégia `cot_visual`, processou 546 ficheiros e não registou erros de execução. Este resultado confirma sobretudo a robustez da pipeline de leitura, processamento, serialização JSON e avaliação automática. As métricas que dependem de *ground truth* fino por *issue*, como *Issue Detection Rate* e *Severity Accuracy*, ficaram sem valor numérico porque o relatório de avaliação local tinha `ground_truth_issues=0`. Por esse motivo, esses campos são assinalados como não aplicáveis no ensaio local e devem ser recalculados quando existir o conjunto anotado manualmente do professor.

B. Métricas

As métricas implementadas ou previstas seguem o enunciado: *Issue Detection Rate*, *False Positive Rate*, *Severity Accuracy*, *JSON Parse Rate* e *Hallucination Rate* para análise visual; *Recall@3*, *Faithfulness* e *Answer Relevance* para RAG; *Rule Parse Rate*, *Rule Correctness* e *Ambiguity Detection* para regras.

A avaliação do Rule Engine incluiu três exemplos representativos: uma regra clara sobre prateleira inferior mais de 30% vazia, uma regra ambígua com a frase «Avisa-me quando a prateleira estiver vazia», e uma regra sobre produto tombado com severidade alta. O sistema converteu as regras para JSON válido e identificou corretamente a ambiguidade principal da segunda regra: não era claro se «vazia» significava 0% de produto ou abaixo de uma percentagem operacional. Esta evidência mostra que o componente não se limita a aceitar qualquer frase; tenta separar regras executáveis de regras que exigem clarificação.

C. LLM-as-Judge

O `evaluate_judge.py` avalia qualitativamente relatórios e respostas RAG. O avaliador recebe o output do sistema, um critério de avaliação e devolve pontuação com justificação. Quando a API Gemini está indisponível, usa `local_fallback`. Esta decisão é importante para a demonstração: o sistema continua a produzir uma avaliação estrutural, mesmo que não consiga chamar o juiz externo.

Tabela III
RESULTADOS DO ENSAIO LOCAL DE AVALIAÇÃO.

Métrica	Valor	Interpretação
Imagens processadas	546/546	Execução completa, sem erros registados.
Issue Detection Rate	N/A	Sem <i>ground truth</i> fino por <i>issue</i> neste ensaio.
False Positive Rate	0.00	Não foram registados falsos positivos no ficheiro local.
Severity Accuracy	N/A	Sem <i>ground truth</i> de severidade neste ensaio.
JSON Parse Rate	1.00	Todas as respostas avaliadas ficaram em JSON parseável.
Hallucination Rate	0.00	Não foram assinaladas alucinações pelo avaliador local.
Recall@3	N/A	Sem queries RAG anotadas com documento esperado.
Rule Parse Rate	1.00	Regras de teste convertidas em JSON válido.
Rule Correctness	1.00	Regras sintéticas executaram de acordo com o esperado.
Ambiguity Detection	1.00	A regra ambígua foi assinalada como inválida/incompleta.

A meta-análise é a seguinte: o juiz automático é útil para verificar presença de secções, aderência ao formato e existência de recomendações; porém, pode sobrevalorizar relatórios bem estruturados mas pouco informativos. A avaliação humana continua essencial para verificar se as recomendações são realmente acionáveis, se o histórico recuperado é relevante e se as conclusões não extrapolam a evidência visual.

X. INTEGRAÇÃO COM DADOS DE TRAJETÓRIA

A integração com o Projeto 1 foi tratada como componente opcional e preparada conceptualmente no relatório. A ideia é cruzar *issues* visuais com dados de afluência por zona e período. Por exemplo, se a zona `Z_S1` teve *fill rate* baixo às 15h e os dados de trajetória indicam afluência 40% acima da média nesse período, a explicação provável pode ser procura elevada. Se, pelo contrário, a zona apresenta baixa afluência e prateleira vazia recorrente, a causa pode ser falha de reposição ou problema de organização.

Esta integração melhora a utilidade operacional porque transforma deteções visuais em hipóteses de causa. No estado atual, o sistema reserva uma secção específica no relatório para essa informação. Quando os ficheiros de trajetória estão disponíveis, podem ser agregados por `zone_id`, hora e dia da semana, e adicionados como contexto ao `report_generator.py`.

XI. ANÁLISE DE FALHAS E LIMITAÇÕES

A. Falhas da análise visual

Os principais casos de falha esperados são:

- **ângulos oblíquos:** dificultam estimar percentagem vazia;
- **iluminação fraca:** pode transformar sombra em falso vazio;
- **produtos semelhantes:** dificultam distinguir produto errado de produto correto;
- **planograma desconhecido:** sem referência externa, o modelo infere apenas por evidência visual;

- **ambiguidade semântica:** uma prateleira parcialmente vazia pode ser normal se estiver em reposição.

B. Falhas do Rule Engine

O Rule Engine depende da clareza da regra. Expressões como «muito vazia», «urgente» ou «zona principal» exigem interpretação. O sistema tenta detetar essas ambiguidades, mas pode aceitar pressupostos quando a frase parece clara. A solução passa por validação humana antes de ativar regras críticas.

C. Falhas do RAG

A recuperação semântica pode devolver inspeções linguisticamente semelhantes mas operacionalmente pouco relevantes. Por exemplo, duas inspeções podem mencionar «prateleira inferior», mas pertencer a zonas e períodos diferentes. A mitigação é usar metadados: zona, data, tipo de *issue* e severidade.

D. Limitações de reprodutibilidade

Modelos Gemini não expõem uma seed determinística universal para todos os cenários. A implementação usa temperatura 0 sempre que possível, mas pequenas variações podem ocorrer entre chamadas. Por isso, os resultados relevantes são persistidos e o cache impede que a mesma imagem seja reavaliada desnecessariamente.

XII. CONCLUSÃO

O Retail Vision Intelligence System cumpre o objetivo de construir uma camada de inteligência visual para retalho, integrando análise multimodal, regras em linguagem natural, memória RAG e geração de relatórios. O que funcionou melhor foi a arquitetura modular: cada componente pode ser testado isoladamente e a interface final consegue orquestrar a pipeline completa. Também funcionou bem a estratégia de *fallback*, porque reduz o risco de falha durante demonstração ao vivo.

O que apresentou maior dificuldade foi a construção e classificação do dataset. Como os datasets públicos nem sempre têm rótulos diretamente compatíveis com as categorias operacionais do enunciado, foi necessário organizar as imagens em classes adaptadas e aceitar uma margem de ambiguidade. Outra dificuldade foi garantir que o relatório final não fosse apenas formalmente correto, mas operacionalmente útil.

Se o projeto fosse prolongado, as melhorias prioritárias seriam: criar ground truth mais fino por região da prateleira, integrar planogramas reais por zona, usar validação humana para calibrar severidade, automatizar o manifesto de dataset e reforçar a integração com dados de trajetória do Projeto 1. Ainda assim, a solução entregue constitui uma pipeline funcional, auditável e demonstrável para inspeção contínua de prateleiras com memória histórica.

APÊNDICE A PROMPTS USADOS

A. Prompt zero-shot

```
You are a retail shelf inspection assistant. Analyze the image and return only valid JSON following the required schema. Detect empty shelves, wrong product placement, damaged products, misalignment, missing labels and other visible issues. Estimate shelf_fill_rate from 0 to 1. Do not include text outside JSON.
```

B. Prompt de raciocínio visual estruturado

```
You are inspecting a retail shelf image. First inspect the top, middle and bottom shelf regions. Then inspect left, center and right areas. Identify visual evidence of empty space, wrong placement, damaged packaging, misalignment or missing labels. After this reasoning summary, output valid JSON only with the required schema.
```

C. Prompt few-shot textual

```
Example 1: A shelf is mostly full, products aligned, no visible issue. Expected status: ok, issues: [].
Example 2: Bottom-left area has large empty spaces, estimated fill rate 0.55. Expected issue type: empty_shelf, severity: high.
Example 3: Several products are tilted and labels are hard to read. Expected issue type: misaligned, severity: medium.
Now analyze the provided image and return valid JSON only.
```

D. Prompt do Rule Engine

```
Convert the manager natural language rule into executable JSON. Preserve the original text. Identify conditions, alert action, ambiguities and assumptions. If the rule is ambiguous, set validation.is_valid to false and list what must be clarified before saving.
```

E. Prompt do Report Generator

```
Generate a Markdown inspection report for a retail store manager. Include: executive summary under 150 words, problems by zone, triggered rules, relevant historical context with inspection_id and date, maximum five concrete recommendations ordered by urgency, and trajectory integration when available. Be direct and operational.
```

APÊNDICE B EXEMPLO DE RELATÓRIO GERADO

Listing 6. Excerto de relatório Markdown esperado.

```
# Inspection Report

## 1. Sumario executivo
Foram inspecionadas 5 zonas. Foram detetados 2 problemas criticos e 3 warnings. A prioridade operacional e repor stock na zona Z_S3 e corrigir desalinhamento na zona Z_S1.

## 2. Problemas por zona
### Z_S3
- Issue: empty_shelf
- Severity: high
- Fill rate: 0.58
- Historico: problema semelhante em INS_20260610_153000.

## 3. Regras disparadas
- RULE_001: fill rate inferior a 60% na prateleira inferior.
```

```
## 4. Contexto historico relevante
- INS_20260610_153000, Z_S3, empty_shelf, high.

## 5. Recomendacoes
1. Repor produtos na prateleira inferior da Z_S3 antes do
   proximo pico de afluencia.
2. Confirmar stock em armazem para a categoria afetada.
```

APÊNDICE C

COMANDOS DE REPRODUÇÃO

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python src/interface.py inspect Z_S1 --image data/images/
normal/train_1.jpg
streamlit run app.py
python evaluate.py --images-dir data/images --output
evaluation_report.json
python evaluate_judge.py
```

APÊNDICE D

CHECKLIST DE CONFORMIDADE

Tabela IV
CHECKLIST FINAL DE CONFORMIDADE COM O ENUNCIADO.

Requisito	Estado
shelf_inspector.py	Implementado
rule_engine.py	Implementado
rag_memory.py	Implementado
report_generator.py	Corrigido
interface.py CLI	Implementado
app.py Streamlit	Implementado
Três estratégias de prompting	Implementado
Cache por MD5	Implementado
Rate limiting/backoff/fallback	Implementado
Dataset com cinco categorias	Organizado
RAG com vector store local	Implementado
Relatório Markdown obrigatório	Implementado
evaluate.py	Implementado
evaluate_judge.py	Implementado

REFERÊNCIAS

- [1] LIACD, «Trabalho Prático #2: Retail Vision Intelligence System», enunciado da unidade curricular *Interação com Modelos de Larga Escala*, ano letivo 2025/2026.
- [2] E. Goldman, R. Herzig, A. Eisenschtat, J. Goldberger e T. Hassner, «Precise Detection in Densely Packed Scenes», CVPR, 2019. Repositório SKU-110K: https://github.com/eg4000/SKU110K_CVPR19.
- [3] M. Klasson, C. Zhang e H. Kjellström, «A Hierarchical Grocery Store Image Dataset with Visual and Semantic Labels», IEEE Winter Conference on Applications of Computer Vision, 2019. Repositório: <https://github.com/marcusklasson/GroceryStoreDataset>.
- [4] Google AI for Developers, «Structured outputs», Gemini API documentation. <https://ai.google.dev/gemini-api/docs/structured-output>.
- [5] Google AI for Developers, «Rate limits», Gemini API documentation. <https://ai.google.dev/gemini-api/docs/rate-limits>.
- [6] Sentence Transformers, «paraphrase-multilingual-MiniLM-L12-v2», Hugging Face model card. <https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2>.
- [7] Chroma, «Introduction», Chroma documentation. <https://docs.trychroma.com/docs/overview/introduction>.